

Metrics in field extraction

Some quick notes

Motivating example

1. If you use this definition from [here](#):

- **True positive (TP)**: The field exists in the ground truth and our system correctly extracted its value according to the field-specific comparator
- **False positive (FP)**: Our system extracted a value for a field, but either the field doesn't exist in the ground truth or the extracted value doesn't match the expected value
- **False negative (FN)**: The field exists in the ground truth, but our system failed to extract it
- **True negative (TN)**: The field doesn't exist in the ground truth, and our system correctly didn't extract it

Motivating example - Invoice number

Document ID	Ground Truth (Expected)	Prediction (Extracted)	Match/Mismatch
doc1	INV-100	INV-100	Match
doc2	INV-200	INV-999	Mismatch
doc3	INV-300	INV-300	Match
doc4	INV-400	INV-888	Mismatch

Motivating example - Invoice number

Document ID	Ground Truth (Expected)	Prediction (Extracted)	Match/Mismatch
doc1	INV-100	INV-100	Match
doc2	INV-188	INV-888	Mismatch
doc3	INV-300	INV-300	Match
doc4	INV-400	INV-888	Mismatch

Is this really a high-recall/low-precision model to you?

Motivating example - Invoice number

Document ID	Ground Truth (Expected)	Prediction (Extracted)	Match/Mismatch
doc1	INV-100	INV-100	Match
doc2	INV-200	INV-999	Mismatch
doc3	INV-300	INV-300	Match
doc4	INV-400	INV-888	Mismatch

- **True positive (TP)**: The field exists in the ground truth and our system correctly extracted its value according to the field-specific comparator
- **False positive (FP)**: Our system extracted a value for a field, but either the field doesn't exist in the ground truth or the extracted value doesn't match the expected value
- **False negative (FN)**: The field exists in the ground truth, but our system failed to extract it
- **True negative (TN)**: The field doesn't exist in the ground truth, and our system correctly didn't extract it

Precision: $TP / (TP + FP) = 2 / (2 + 2) = 50\%$

Recall: $TP / (TP + FN) = 2 / (2 + 0) = 100\%$

Motivating example - Invoice number

Document ID	Ground Truth (Expected)	Prediction (Extracted)	Match/Mismatch
doc1	INV-100	INV-100	Match
doc2	INV-200	INV-999	Mismatch
doc3	INV-300	INV-300	Match
doc4	INV-400	INV-888	Mismatch

- **True positive (TP)**: The field exists in the ground truth and our system correctly extracted its value according to the field-specific comparator
- **False positive (FP)**: Our system extracted a value for a field, but either the field doesn't exist in the ground truth or the extracted value doesn't match the expected value
- **False negative (FN)**: The field exists in the ground truth, but our system failed to extract it
- **True negative (TN)**: The field doesn't exist in the ground truth, and our system correctly didn't extract it

Precision: $TP / (TP + FP) = 2 / (2 + 2) =$
50%

Recall: $TP / (TP + FN) = 2 / (2 + 0) =$
100%

“Our model has low precision, but high recall for invoice number” $\Rightarrow$ Gives the impression predictions from the model will *usually* contain the right invoice number somewhere

Motivating example - Invoice number

Document ID	Ground Truth (Expected)	Prediction (Extracted)	Match/Mismatch
doc1	INV-100	INV-100	Match
doc2	INV-200	INV-999	Mismatch
doc3	INV-300	INV-300	Match
doc4	INV-400	INV-888	Mismatch

- **True positive (TP)**: The field exists in the ground truth and our system correctly extracted its value according to the field-specific comparator
- **False positive (FP)**: Our system extracted a value for a field, but either the field doesn't exist in the ground truth or the extracted value doesn't match the expected value
- **False negative (FN)**: The field exists in the ground truth, but our system failed to extract it
- **True negative (TN)**: The field doesn't exist in the ground truth, and our system correctly didn't extract it

$$\text{Precision: } TP / (TP + FP) = 2 / (2 + 2) = 50\%$$

$$\text{Recall: } TP / (TP + FN) = 2 / (2 + 0) = 100\%$$

BAD

~~"Our model has low precision, but high recall for invoice number" $\Rightarrow$ Gives the impression predictions from the model will *usually* contain the invoice number somewhere~~

Why is it bad?

1. Optimising recall using those definitions $\Rightarrow$ Optimising for “detecting” invoice_number but not actually extracting it correctly

What is a better approach?

1. Insight - a mistake in field extraction should increment *both* FN and FP
 - a. Not only did you not detect the correct value, but you wrongly guessed the other value.
2. New definitions:
 - a. gt_correct: Field value is in the ground truth, model predicted something, and it is correct
 - b. gt_incorrect: Field value is in the ground truth, model predicted something, but it is incorrect
 - c. gt_missing: Field value is in the ground truth, model predicted nothing
 - d. nogt_incorrect: Field value not in ground truth, model predicted something (so is incorrect)
 - e. nogt_correct: Field value not in ground truth, model predicted nothing (so it is correct)
3. New calculations
 - a. $FP = gt_incorrect + nogt_incorrect$
 - b. $FN = gt_incorrect + gt_missing$
 - c. $TP = gt_correct$
 - d. $TN = nogt_correct$

Other non-obvious lessons

1. Treat these two cases as the same: Model detected a field was there and predicted “None” as the value & model completely missed detecting the field
 - a. Why? Because no one cares about the difference - a client/end-user will hardly/never require you to make a distinction
 - b. Often a sign of premature optimization - trying to keep these two cases distinct will 100% make your code horrible.